K17: a UTXO-chained stack machine with a liveness auction and witness-verified bounties on Bitcoin SV

Technical disclosure, published to establish prior art.

- **Author / inventor:** Jerry David Chan (Dave), Tokyo, Japan.

- **Date of disclosure:** 2026-10-18.
- **Reference implementation:** repository `k17-vm`, commit `9b2733fffe6f3771b5b538ec127173be2f37b3af` (2026-10-18 13:38:44 +0900). SHA-256 of `git archive --format=tar HEAD` at that commit: `8c73d7808652fc6de853b615923091f26cc315b7fedc0e21416368979b3f34f6`.

- **Licence of this document:** Creative Commons Attribution 4.0 International (CC BY 4.0). The source code carries the repository's MIT licence.

- **Purpose:** this document places every mechanism below in the public domain of prior art as of the date above. The author makes no claim that any of it is free of third-party patents; see section 9.

---

1. Abstract

A single unspent transaction output (UTXO) on a Bitcoin-style ledger is made to hold one suspended process of a small stack-machine. The UTXO's locking script contains a fixed interpreter for a 28-instruction bytecode, derived from Koopman's Canonical Stack Machine, and a data tail holding the program, the program counter, two stacks, a memory image, an output log and a timestamp. Anyone may spend the UTXO. The spend is valid only if the new output carries byte-identical code plus the state advanced by up to K instructions, and if the spender takes no more than a reward that grows with the time elapsed since the previous cycle, measured from the transaction's nLockTime. Illegal operations halt the machine and return its funds rather than invalidating the transaction, so no program can freeze its own escrow. An extension lets programs demand witness values from the spender, reject wrong values at the consensus layer, release escrowed bounties to the party that supplied correct values, end a cycle early so that a multi-instruction stage executes atomically, and bind a commitment to the eventual payee's address so that a revealed answer cannot be copied out of the mempool by a third party.

2. Field

Bitcoin-script covenants that carry state across chained transactions; incentive design for permissionless state advancement ("keeper" markets); escrowed payment for verifiable work; mitigation of transaction front-running.

3. Background and acknowledged prior art

The following is known and is not claimed as new by this disclosure:

- Carrying contract state in a locking script and validating the transition by inspecting the spending transaction's signature preimage (the "OP_PUSH_TX" technique), as used by sCrypt stateful contracts since 2020 and by the STAS token templates.
- A bytecode interpreter embodied in a locking script, with the program and interpreter replicated into the next output and execution-state data, including execution pointers, carried in the locking script: nChain, US 12,020,249 B2 and continuations (priority 2017-08-15); nChain, US 11,915,238 B2 (concurrent bytecode threads). The description of that family also mentions constraining the number of bytecodes per transaction and calculating a reward for the entity creating the next transaction.
- One-step-per-transaction Turing machines and cellular automata on Bitcoin SV (sCrypt, 2020 to 2023).
- Commit-reveal with the committer's address inside the hash to defeat front-running (common Ethereum practice; Canidio and Danos 2023; the 2019 front-running SoK; ZKCP literature).
- Bounties with proposer commitments, deposits and verification games (Truebit 2017; nChain US 11,509,455 B2; BitVM and BitVMX).
- Keeper networks (MakerDAO, Keep3r, Gelato, Chainlink Automation) and descending-price Dutch auctions for liquidations.

Koopman, *Stack Computers: The New Wave* (1989), chapter 3, supplies the 17-instruction Canonical Stack Machine and chapter 6 the stack-depth measurements that justify 32-cell stacks.

4. The machine

4.1 State

Field	Type	Meaning
`prog`	bytes, immutable	program bytecode
`pc`	integer	byte offset into `prog`
`ds`	bytes	data stack, 4-byte cells, top at the end
`rs`	bytes	return stack, 4-byte cells, top at the end
`mem`	bytes	64 cells of memory
`outLog`	bytes	append-only output cells, maximum 64
`lastTime`	integer	nLockTime of the transaction that produced this state

Immutable parameters: `beneficiary` (20-byte address hash), `rewardMin`, `slope`, `rewardMax` (satoshis, satoshis per second, satoshis).

Cells are unsigned 32-bit integers, little-endian, arithmetic modulo 2^32. In Bitcoin script a cell is produced by `OP_NUM2BIN` to 5 bytes followed by `OP_SPLIT` at 4, and read by appending `0x00` and `OP_BIN2NUM`, so that cells are uniform and never carry a sign byte.

4.2 Instruction set

Byte	Mnemonic	Stack effect	Semantics
00	HALT		stop; release funds (section 4.5)
01	!	N ADDR →	mem[ADDR] = N; fault if ADDR ≥ 64
02	+	A B → A+B	
03	-	A B → A−B	
04	>R	N →	push N on return stack

Byte	Mnemonic	Stack effect	Semantics
05	@	ADDR → mem[ADDR]	fault if ADDR ≥ 64
06	AND	A B → A∧B	
07	DROP	N →	
08	DUP	N → N N	
09	OR	A B → A∨B	
0A	OVER	A B → A B A	
0B	R>	→ N	pop return stack
0C	SWAP	A B → B A	
0D	XOR	A B → A⊕B	
0E	[IF] n	F →	if F = 0 then pc = n
0F	[CALL] n		push pc_after on RS; pc = n
10	[EXIT]		pc = pop RS
11	[LIT] n	→ n	
12	0=	N → (N=0)	
13	OUT	N →	append N to outLog; fault if full
14	TIME	→ T	T = nLockTime of this transaction
15	IN	→ W	next witness cell from the unlocking script; **transaction invalid** if none left
16	VERIFY	F →	**transaction invalid** if F = 0
17	BOUNTY n		authorise n extra satoshis to the spender in this transaction

Byte	Mnemonic	Stack effect	Semantics
18	*	A B → A·B	
19	HASH	N → H	H = first 4 bytes of SHA-256 over the 4-byte cell N
1A	WHO	→ A	A = first 4 bytes of the spender's payout address hash
1B	YIELD		end this cycle after this instruction

Opcodes 0E, 0F, 11 and 17 carry a 4-byte little-endian operand. Instructions 00 to 11 are Koopman's Table 3.1 verbatim.

4.3 Faults

Stack underflow, stack overflow beyond 32 cells, a memory address ≥ 64, an unknown opcode, a program counter past the end of the program, and an output log past 64 cells are **faults**. A fault is treated exactly as HALT: the machine stops and the transaction pays out as in section 4.5. A fault never invalidates the transaction. This guarantees that an escrow can always be released and that no program, however buggy, can be made unspendable by its own execution.

Two outcomes are deliberately **not** faults: `IN` with no witness left and `VERIFY` of zero. Both make the spending transaction invalid. The consequence is that a program waiting for an answer cannot be advanced by a party without one, and a wrong answer is rejected rather than terminating the program.

4.4 The cycle transaction

Input 0 is the machine UTXO. Its unlocking script carries the signature preimage, the spender's payout address hash and the witness cells. Further inputs pay the mining fee.

Output 0 carries the identical interpreter and program with the advanced state (or, if the machine halted, a pay-to-public-key-hash to `beneficiary`). Output 1 pays the spender. Output 2 is the spender's change. The script rebuilds outputs 0 and 1, appends the change output, and requires `hash256(outputs)` to equal the `hashOutputs` field of the preimage.

The interpreter executes up to K instructions (K = 40 in the reference implementation) by unrolling K copies of the decode-and-dispatch block. Execution of a cycle stops early on HALT, on a fault, or on YIELD.

4.5 Payout rule

```
assert sequence < 0xffffffff            // nLockTime is enforced by consensus
now     = nLockTime (from the preimage); assert now ≥ 500 000 000 (a unix time)
elapsed = now − lastTime;              assert elapsed ≥ 0
reward  = min(rewardMin + slope·elapsed, rewardMax) + bountyDue
reward  = min(reward, value − 1)
if halted: output0 = P2PKH(beneficiary, value − reward)
else:      output0 = this script with new state and lastTime = now, value − reward
if reward > 0: output1 = P2PKH(spender, reward)
```

4.6 The liveness Dutch auction

Because nLockTime cannot be set later than the real time at which a transaction becomes mineable (miners enforce it against median time past), `elapsed` is a consensus-enforced lower bound on the silence since the previous cycle. A keeper who wants a larger reward must wait longer before its transaction can be mined. Cheap keepers therefore advance the machine early for a small reward; expensive keepers only win when cheap ones are absent. The machine buys liveness at the marginal cost of the cheapest live operator, and redundancy appears only when needed. The reward is an increasing function of elapsed time; the reference implementation uses a linear function with a cap. Convex, exponential, and deadline-anchored schedules are alternative embodiments.

4.7 Uniqueness rule

The script requires its own outpoint to be at index 0 of the serialized `prevouts`, checked against `hashPrevouts`. Without this, two machines with byte-identical scripts and equal values could be spent in one transaction, both accepting a single shared state output while the spender collects the second balance as change. Requiring index 0 means at most one machine per transaction.

5. Witness-verified bounties

A program stage of the form

```
YIELD  IN ... IN  <checks using VERIFY>  BOUNTY n  <side effects>  YIELD
```

executes atomically in one transaction: the leading YIELD makes the previous cycle stop at the stage boundary; `IN` prevents any party without witnesses from entering the stage; `VERIFY` rejects wrong witnesses; `BOUNTY n` releases n satoshis to the spender of that same transaction; the

trailing YIELD stops the cycle before the next stage. The party that supplied the witnesses is therefore necessarily the party that is paid. Dependent stages, where stage 2's acceptance test depends on stage 1's answer stored in memory, follow directly.

5.1 Example: factoring bounty

```
YIELD
IN IN                                   ; p q
OVER LIT 0xFFFF0000 AND 0= VERIFY       ; p < 2^16
DUP  LIT 0xFFFF0000 AND 0= VERIFY       ; q < 2^16  (so p·q cannot wrap)
OVER LIT 1 - VERIFY                     ; p ≠ 1
DUP  LIT 1 - VERIFY                     ; q ≠ 1
OVER OVER * LIT N - 0= VERIFY           ; p·q = N
BOUNTY 40000
OUT OUT HALT
```

5.2 Payee-bound commit-reveal

To stop a mempool observer copying a revealed answer x and re-broadcasting it with its own payout address, the reveal is split into two transactions:

- commit: `IN` c, where c = HASH(x ⊕ WHO) computed by the solver with its own address prefix; store c and TIME.
- reveal: `IN` x; `DUP WHO XOR HASH <c> - 0= VERIFY` checks the commitment against the **current** spender's WHO; then the puzzle check; then BOUNTY.

A copier who changes the payout address changes WHO, so the commitment check fails. A copier who keeps the solver's address pays the solver.

5.3 Commit expiry and deadline

A commit may be overwritten once `commitTtl` seconds have passed since it was made, so a bogus commit cannot block the stage forever. The TTL must exceed worst-case confirmation latency (hours, not minutes) because an honest reveal left unconfirmed past the TTL exposes x to an overwriter. Once TIME reaches a global `deadline` the program HALTs and the escrow returns to the beneficiary. These are program-level mechanisms; a committer bond and append-only commitments are alternative embodiments.

6. Off-chain components

A keeper or solver (a) reads the state from the UTXO, (b) runs a reference implementation of the same semantics for K instructions with its chosen nLockTime, payout address and witness cells, (c) computes the permitted reward, (d) builds the transaction with the state output, the reward

output and change, and (e) broadcasts it. The reference VM and the on-chain interpreter are kept in agreement by differential tests: every accepted cycle in the test suite is computed off-chain and then verified by a Bitcoin script interpreter.

7. Measurements (reference implementation)

K	Handlers	Locking script	Fibonacci(10), 247 instructions
1	21	9 026 bytes	247 cycles
4	21	16 247 bytes	62 cycles
40	28	159 003 bytes	7 cycles

About 2.4 to 3.8 KB of script per unrolled instruction plus about 6.6 KB fixed. Test programs used reward parameters of 100 to 500 satoshis, which are below realistic mining fees; production parameters must exceed the fee of the cycle transaction.

8. Alternative embodiments

1. Variable cycle length chosen by the spender up to K, so that heartbeats do not pay for long transactions.
2. Any monotone reward schedule in elapsed time, including convex, exponential, step, and schedules anchored to a deadline.
3. Rewards and bounties paid to an arbitrary script rather than a public-key hash; WHO computed over the full payee script rather than a 4-byte prefix.
4. Full-width HASH over a memory range; signature verification, Merkle-inclusion and zero-knowledge-proof verification as instructions, so that acceptance tests may be oracle attestations, SPV data or proofs.
5. Memory stored as a Merkle root with inclusion proofs supplied per cycle.
6. Committer bonds forfeited on non-reveal; append-only commitments; a reveal window measured from the commit.
7. Gated variants: spender must belong to a registered set, or hold a bond, or be selected round-robin from a hash of the previous transaction id.
8. Multiple machines per transaction with each machine bound to a distinct input index and output index.
9. Use as the single-step verifier in an optimistic (challenge-response) execution scheme: the same bytecode runs off-chain, only a disputed step is re-executed on chain.

10. Any ledger whose scripting language can inspect the spending transaction's preimage or equivalent (covenant-capable chains), not only Bitcoin SV.
11. Interpreters for other instruction sets, register machines included, hosted by the same cycle, payout, auction and bounty rules.
12. Application programs: escrows with dead-man return, vesting and subscription drips, timed auctions, heartbeat and uptime logs, multi-stage outsourced-computation protocols, agent-to-agent job markets with on-chain acceptance tests.

9. Freedom-to-operate note

The author is aware that independent claim 1 of US 12,020,249 B2 (nChain Licensing AG) describes a spending transaction and a previous transaction that share the same bytecode sequence and the same interpreter code, with state data in the locking scripts, and that the reference implementation has that structure. Nothing in this disclosure is a representation that practising it is free of that or any other patent. The disclosure's purpose is to prevent later patents on the specific mechanisms in sections 4.3, 4.5 to 4.7 and 5, not to assert rights.

10. Verification of this disclosure

The reference implementation at the commit named above contains 29 tests; 10 cover the reference VM and 19 run cycle transactions through a Bitcoin script interpreter, including the factoring bounty, the two-stage commit-reveal job with a refused front-runner, commit overwrite after expiry, deadline refund, the uniqueness rule, and a random-program differential fuzz. `npm test` reproduces them with no network access.